Notebook Export: Kreas.ipynb

Rendered from saved notebook outputs

Cell 1 - code

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
import matplotlib.pyplot as plt
```

Cell 2 - code

```
(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()

print("Training images:", x_train.shape)
print("Test images:", x_test.shape)
```

Output 2.1

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
□[1m11490434/11490434□[0m □[32m━━━━━━━━━━━━━━━━━━━━□[0m□[37m□[0m □[1m2s□[0m 0us/step
Training images: (60000, 28, 28)
Test images: (10000, 28, 28)
```

Cell 3 - code

```
x_train = x_train / 255.0
x_test = x_test / 255.0
```

Cell 4 - code

```
x_train = x_train.reshape(-1, 784)
x_test = x_test.reshape(-1, 784)

print(x_train.shape)
```

Output 4.1

```
(60000, 784)
```

Cell 5 - code

```
model = keras.Sequential([

    layers.Dense(128, activation='relu', input_shape=(784,)),  # Hidden layer

    layers.Dropout(0.2),  # Dropout layer

    layers.Dense(10, activation='softmax')  # Output layer

])
```

Output 5.1

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)`
object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Cell 6 - code

```
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

Cell 7 - code

```
history = model.fit(
    x_train,
    y_train,
    epochs=10,
    validation_data=(x_test, y_test)
)
```

Output 7.1

```
Epoch 1/10
[1m1875/1875[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m9s[0m 4ms/step - accuracy: 0.9129 - loss:
0.2993 - val_accuracy: 0.9598 - val_loss: 0.1363
Epoch 2/10
[1m1875/1875[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m7s[0m 4ms/step - accuracy: 0.9581 - loss:
0.1419 - val_accuracy: 0.9700 - val_loss: 0.1005
Epoch 3/10
[1m1875/1875[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m7s[0m 4ms/step - accuracy: 0.9674 - loss:
0.1081 - val_accuracy: 0.9757 - val_loss: 0.0806
Epoch 4/10
[1m1875/1875[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m8s[0m 4ms/step - accuracy: 0.9729 - loss:
0.0880 - val_accuracy: 0.9788 - val_loss: 0.0720
Epoch 5/10
[1m1875/1875[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m7s[0m 4ms/step - accuracy: 0.9769 - loss:
0.0746 - val_accuracy: 0.9783 - val_loss: 0.0774
Epoch 6/10
[1m1875/1875[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m8s[0m 4ms/step - accuracy: 0.9796 - loss:
0.0649 - val_accuracy: 0.9787 - val_loss: 0.0673
Epoch 7/10
[1m1875/1875[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m7s[0m 4ms/step - accuracy: 0.9824 - loss:
0.0564 - val_accuracy: 0.9780 - val_loss: 0.0744
Epoch 8/10
[1m1875/1875[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m8s[0m 4ms/step - accuracy: 0.9823 - loss:
0.0539 - val_accuracy: 0.9797 - val_loss: 0.0684
Epoch 9/10
[1m1875/1875[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m7s[0m 4ms/step - accuracy: 0.9841 - loss:
0.0479 - val_accuracy: 0.9790 - val_loss: 0.0698
Epoch 10/10
[1m1875/1875[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m8s[0m 4ms/step - accuracy: 0.9858 - loss:
0.0432 - val_accuracy: 0.9810 - val_loss: 0.0702
```

Cell 8 - code

```
test_loss, test_acc = model.evaluate(x_test, y_test)

print("Test Accuracy:", test_acc)
```

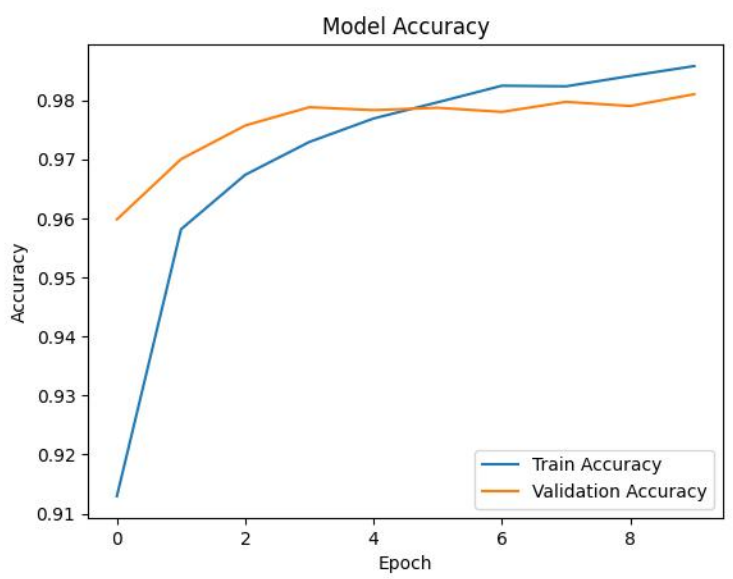
Output 8.1

```
[1m313/313[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m1s[0m 2ms/step - accuracy: 0.9810 - loss: 0.0702
Test Accuracy: 0.9810000061988831
```

Cell 9 - code

```
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')

plt.title("Model Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.show()
```

Output 9.1

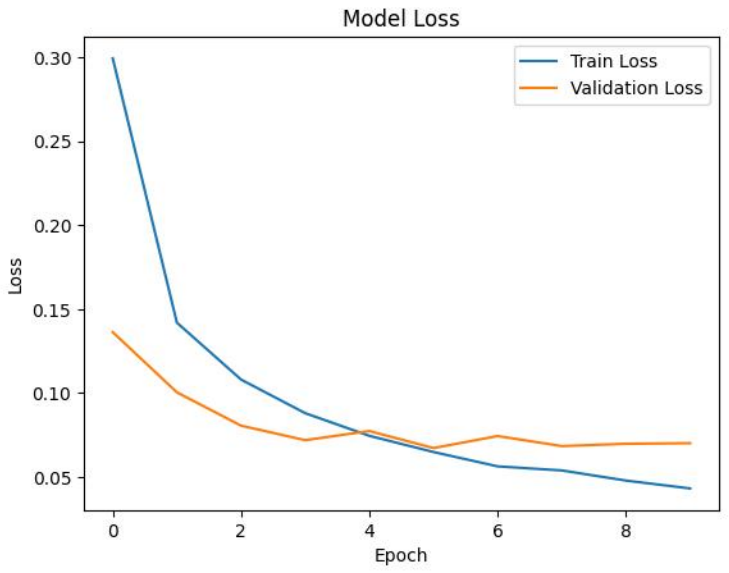



```
<Figure size 640x480 with 1 Axes>
```

Cell 10 - code

```
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')

plt.title("Model Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.show()
```

Output 10.1




```
<Figure size 640x480 with 1 Axes>
```

Cell 11 - code

```
import numpy as np

index = 1   # choose any test image index

prediction = model.predict(x_test[index:index+1])
predicted_label = np.argmax(prediction)

print("Predicted Digit:", predicted_label)
print("Actual Digit:", y_test[index])
```

Output 11.1

```
□[1m1/1□[0m □[32m━━━━━━━━━━━━━━━━━━━━□[0m□[37m□[0m □[1m0s□[0m 36ms/step
Predicted Digit: 2
Actual Digit: 2
```

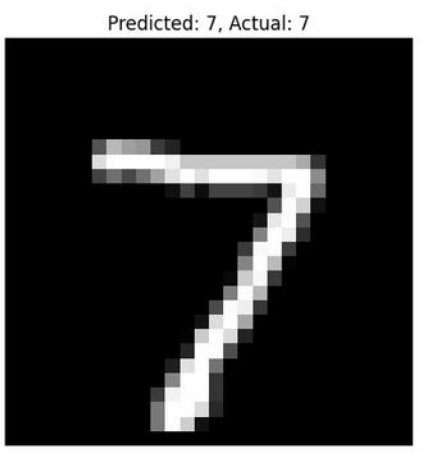
Cell 12 - code

```
for i in range(5):
    prediction = model.predict(x_test[i:i+1])
    predicted = np.argmax(prediction)

    plt.imshow(x_test[i].reshape(28,28), cmap="gray")
    plt.title(f"Predicted: {predicted}, Actual: {y_test[i]}")
    plt.axis("off")
    plt.show()
```

Output 12.1

```
□[1m1/1□[0m □[32m━━━━━━━━━━━━━━━━━━━━□[0m□[37m□[0m □[1m0s□[0m 34ms/step
```

Output 12.2

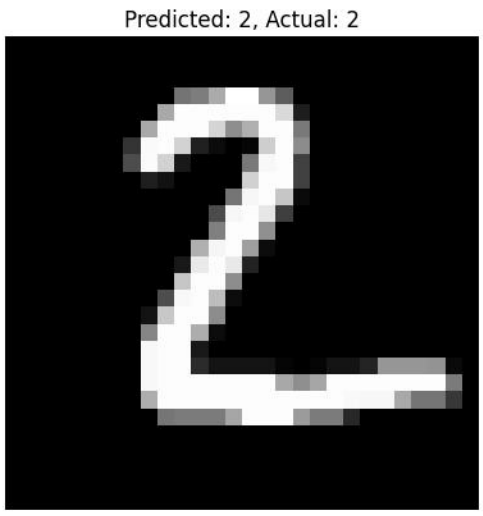


```
<Figure size 640x480 with 1 Axes>
```

Output 12.3

```
[1m1/1[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m0s[0m 38ms/step
```

Output 12.4

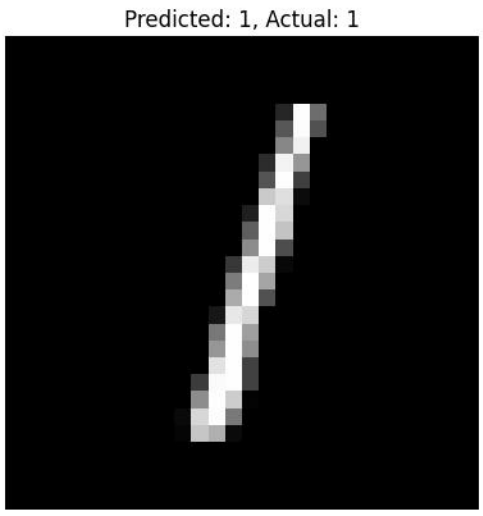



```
<Figure size 640x480 with 1 Axes>
```

Output 12.5

```
[1m1/1[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m0s[0m 34ms/step
```

Output 12.6

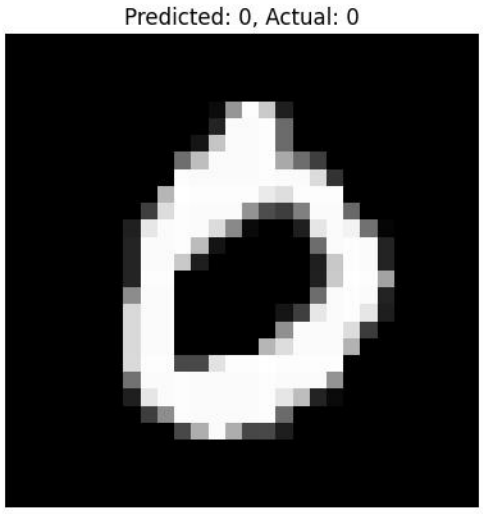



```
<Figure size 640x480 with 1 Axes>
```

Output 12.7

```
[1m1/1[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m0s[0m 36ms/step
```

Output 12.8

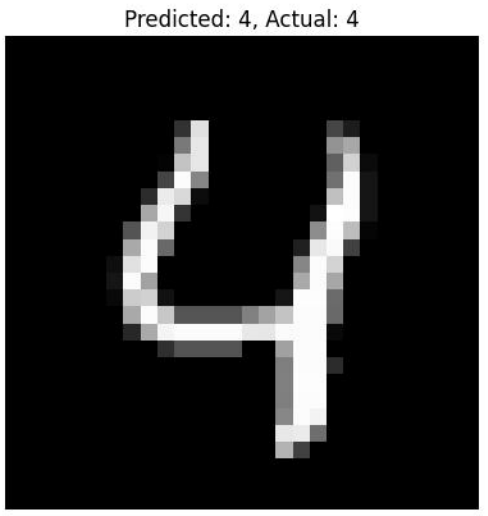



```
<Figure size 640x480 with 1 Axes>
```

Output 12.9

```
[1m1/1[0m [32m━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m0s[0m 35ms/step
```

Output 12.10

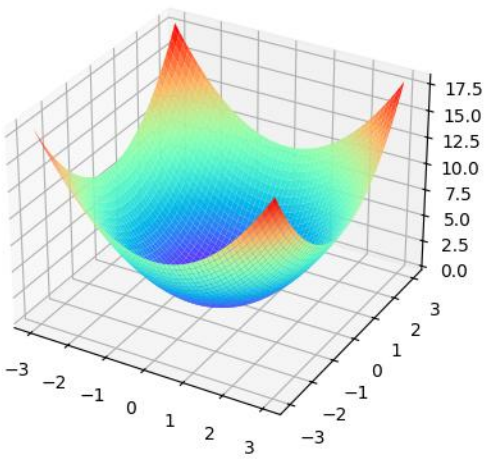



```
<Figure size 640x480 with 1 Axes>
```

Cell 13 - code

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

w0 = np.linspace(-3,3,100)
w1 = np.linspace(-3,3,100)

W0, W1 = np.meshgrid(w0,w1)

Loss = W0**2 + W1**2

fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
ax.plot_surface(W0, W1, Loss, cmap='rainbow')

plt.show()
```

Output 13.1




```
<Figure size 640x480 with 1 Axes>
```